Back to the basics

What is parallelism?

Parallelism simply means performing tasks simultaneously. Parallel computing uses multiple processing elements, wherein each processing element executes a part of the complex large problem simultaneously. The aim of parallel computing is to achieve faster computational performance with efficient utilisation of available resources like CPUs or GPUs (muti-processors/multi cores), memory (registers/cache/RAM), etc. It can be done at different levels which brings us to its types: data parallelism, task parallelism, bit-level, instruction level, and hybrid parallelism.

Data parallelism involves processing large data sets by applying the same function across subsets. It involves synchronous processing where performance scales with the amount of data processed — more data means better performance. This approach is very apt in scenarios where the data under processing is homogenous and data dependency between the operations is not involved with previous operations. It's mostly used in media data like audio/video processing, machine learning and distributed databases.

Task parallelism breaks down large tasks into smaller, parallel-executable subtasks. In contrast with data parallelism, task parallelism is characterised by asynchronous processing, which necessitates managing concurrency (for interdependent data). For instance, multi-threading is an important form of parallelism, wherein programs are divided into threads (small tasks) that share single or multiple cores and other resources like caches and buffers.

Bit-level parallelism refers to the evolution of processor word length. Historically, processor support has progressed from 8-bit to 16-bit, to 32-bit, and now to 64-bit processors. Bit-level parallelism is the capability to process a larger word length in a single instruction or cycle. The more support there is at the bit level, the fewer instructions are required to process data. Bit-level parallelism is commonly used in digital signal processing, cryptography, and arithmetic calculations.

Instruction-level parallelism allows for the execution of multiple instructions simultaneously. For any instruction to run, it typically goes through fetch, decode, and execute steps. Using pipelining techniques, these steps can be parallelised, enabling simultaneous processing of instructions. There are sophisticated branch prediction techniques that enable the pre-fetching of the next instruction well ahead of its execution to improve instruction processing.

Hybrid parallelism is a blend of all these elements. Based on specific needs, one can combine data parallelism and task parallelism with instruction-level parallelism. The end user's experience can be optimised integrating the strengths of each type of parallelism.

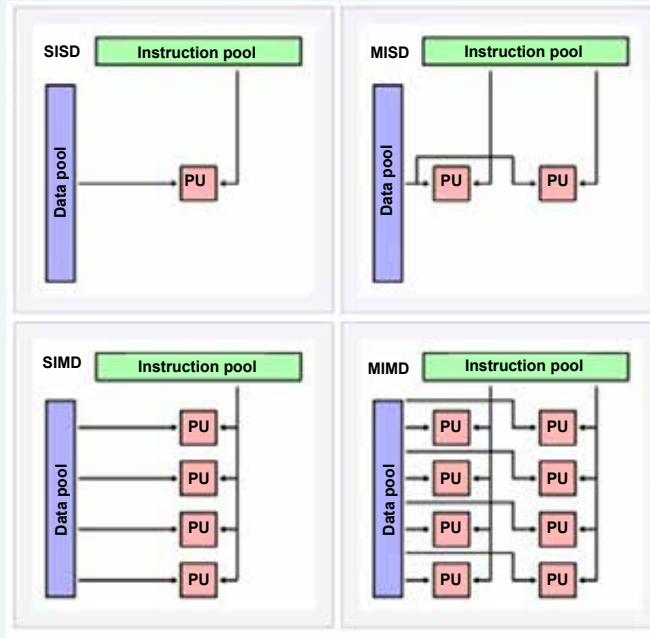

Figure 1: PU -- processing unit (Image credits: Wikipedia --*https://en.wikipedia.org/wiki/Flynn%27s_taxonomy%23Diagram_comparing_classifications*)

Flynn's taxonomy

Based on how instructions and data are used in parallel, Flynn's taxonomy categorises computer architectures into four primary types: SISD, MISD, SIMD, and MIMD. This framework illustrates how different architectures approach parallel processing.

SISD (single instruction, single data stream) involves applying a single instruction to a single data stream to produce one result. It's good for primitive uniprocessors, which execute instructions sequentially.

MISD (multiple instruction, single data stream) involves executing multiple instructions on a single data set. Its practical applications are very limited. MISD can be used in contexts like evaluating fault tolerance, where multiple instructions are executed on the same data to verify the correctness of the results.

SIMD (single instruction, multiple data) involves a single operation being performed on multiple sets of data, known as data parallelism. Here the instruction is broadcast to all processing units, and each unit independently performs the same operation on different pieces of data simultaneously. All modern processors and GPUs have support for this kind of processing using vector instructions extensions. SIMD processors are often referred to as vector processors.

MIMD (multiple instruction, multiple data) involves applying multiple instruction operations to multiple data points within a data set. Supported by many of the latest processors, this approach leverages computer hardware to use multiple instructions on a larger data set.